Cómo estructuro un sistema de agentes

Revisado el 17 de octubre de 2026

TRES VOCES, SIEMPRE SEPARADAS

● LO QUE HICE	◆ LO QUE EL CAMPO HACE	▲ MI POSICIÓN
Sale solo del corpus de este sitio.	Sale de una fuente con título, editor, fecha y enlace.	Es mi criterio, no un hecho.

Lo que no lleva rótulo es mi método: mi posición sobre cómo se hace.

BLOQUE 1

La pregunta

¿Cómo diseñas un sistema multiagente que funcione en producción, y cómo sabes que funciona?

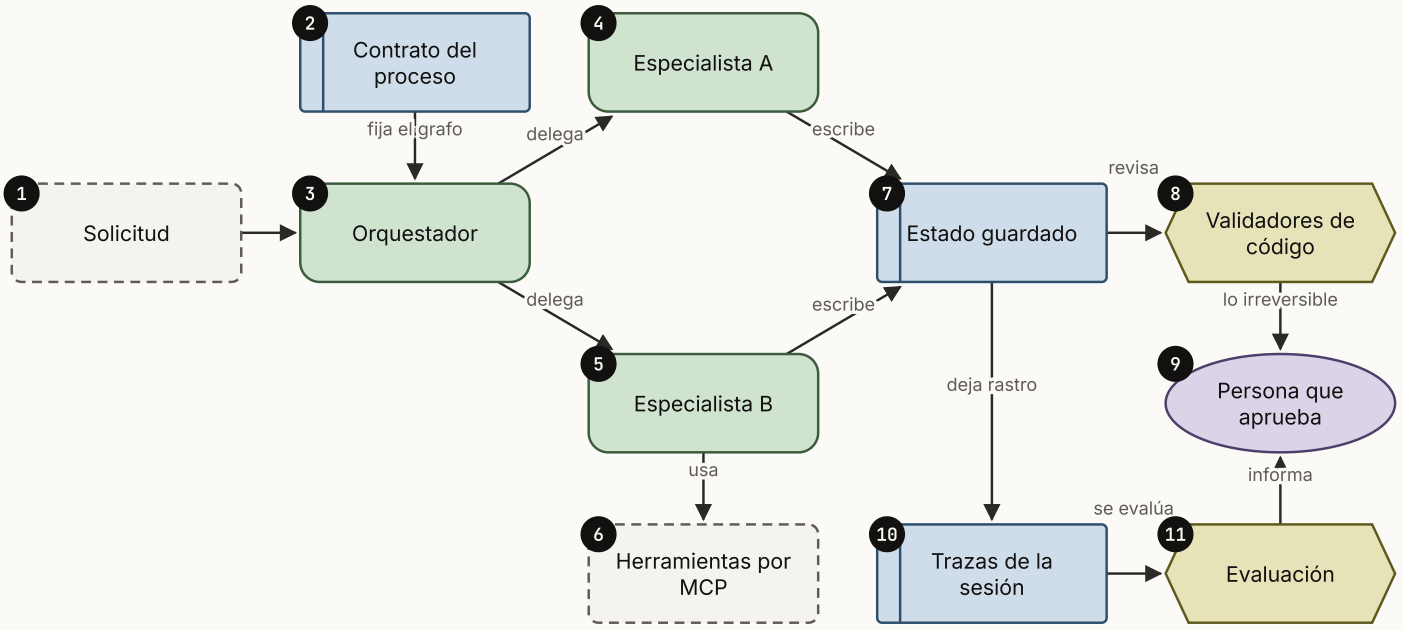
LA RESPUESTA EN TRES LÍNEAS

1. Estructuro un sistema multiagente desde el proceso y no desde el framework: cada actividad se especifica con entradas, función, salidas y excepciones, y solo se vuelve agente la que necesita interpretar; lo demás es código.
2. Reparto el trabajo entre especialistas con su propio contexto, guardo el estado en cada paso y doy una autonomía proporcional al riesgo: lo que no se puede deshacer lo aprueba una persona.
3. Sé que funciona porque lo evalúo antes de ponerlo en producción y después: criterios que pasan o no pasan, un caso roto a propósito para cada control y la sesión entera observada en cuatro dimensiones.

BLOQUE 2

El mapa

El sistema de un vistazo: de la solicitud a la persona que aprueba



LEYENDA DE FORMAS Y COLORES

Actor — quien hace el trabajo

Dato — lo que se guarda y se lee

Control — una regla que se comprueba

Persona — una persona decide

Externo — lo que viene de afuera

LECTURA DEL MAPA, NODO POR NODO

Una solicitud llega al orquestador, que sigue el grafo fijado por el contrato del proceso y delega en dos especialistas, cada uno con su propio contexto. Uno de ellos usa herramientas conectadas por MCP, con permisos mínimos. Los dos escriben en un estado que se guarda paso a paso; validadores de código lo revisan, y lo que no se puede deshacer pasa a una persona que aprueba. Cada paso deja rastro en las trazas de la sesión, que se evalúan y le informan a esa misma persona.

Nodo 1 **Solicitud** (Externo) — Lo que llega de afuera: una pregunta, un caso, un documento por procesar.

Nodo 2 **Contrato del proceso** (Dato) — El proceso dibujado en BPMN con cada actividad abierta: qué recibe, qué hace, qué entrega y cuándo se detiene. De aquí salen el grafo y los criterios.

Nodo 3 **Orquestador** (Actor) — Sigue el grafo, reparte el trabajo y junta los resultados; no hace el trabajo de los especialistas.

Nodo 4 **Especialista A** (Actor) — Trabaja dentro de su harness —el envoltorio que le fija función, fuentes, herramientas y contrato de entrada y salida— y recibe solo el contexto que su tarea necesita.

Nodo 5 **Especialista B** (Actor) — Otro especialista con su propio harness; este además usa herramientas externas.

Nodo 6 **Herramientas por MCP** (Externo) — Fuentes y acciones conectadas por un protocolo común (MCP), habilitadas solo si la tarea las necesita y con los permisos mínimos.

Nodo 7 **Estado guardado** (Dato) — Cada paso se guarda: el trabajo se retoma tras una falla o una pausa. La memoria larga guarda decisiones y lecciones, no conversaciones enteras.

Nodo 8 **Validadores de código** (Control) — Lo que exige exactitud lo comprueba código: un esquema, un cálculo, una cita que debe existir. El modelo no se califica a sí mismo.

Nodo 9 **Persona que aprueba** (Persona) — Aprueba con una palabra exacta lo que no se puede deshacer; sin esa palabra, el sistema no sigue.

Nodo 10 **Trazas de la sesión** (Dato) — La sesión completa: qué se pidió, con qué contexto, qué respondió cada especialista, cuánto tardó, cuánto costó y en qué estado terminó.

Nodo 11 **Evaluación** (Control) — Criterios binarios y casos rotos a propósito antes de publicar; después, la conversación entera y el camino paso a paso.

CONEXIONES

de Solicitud a Orquestador

de Contrato del proceso a Orquestador (fija el grafo)

de Orquestador a Especialista A (delega)

de Orquestador a Especialista B (delega)

de Especialista B a Herramientas por MCP (usa)

de Especialista A a Estado guardado (escribe)

de Especialista B a Estado guardado (escribe)

de Estado guardado a Validadores de código (revisa)

de Validadores de código a Persona que aprueba (lo irreversible)

de Estado guardado a Trazas de la sesión (deja rastro)

de Trazas de la sesión a Evaluación (se evalúa)

de Evaluación a Persona que aprueba (informa)

BLOQUE 3

Las fases

1

FASE 1

Especificar el proceso

ENTRADAS

- El proceso del negocio, dibujado en BPMN.
- La decisión o la acción que el sistema debe habilitar.

ACTIVIDADES

- Abrir cada actividad: entradas, función, salidas, fuentes, herramientas y cuándo detenerse.
- Escribir las excepciones: falta información, hay contradicción, una herramienta no responde.
- Separar lo que necesita interpretar (agente) de lo que exige exactitud (código).

ARTEFACTOS

- El contrato de cada actividad.
- La lista de excepciones con su respuesta.

QUIÉN DECIDE

El responsable del proceso, con quien construye.

CONTROL QUE CIERRA LA FASE

Cada actividad traza a un elemento del proceso; ninguna queda sin dueño.

> • LO QUE HICE
>
> En Vesting esta descomposición —entradas, reglas, fuentes, herramientas, salidas y cuándo pedir a una persona— fue la antesala del proceso core de once etapas con el que se construyeron 27 agentes.

2

FASE 2

Repartir y fijar la autonomía

ENTRADAS

- El contrato de cada actividad.

ACTIVIDADES

- Decidir si basta un agente o hace falta un grafo de especialistas.
- Asignar a cada actividad un nivel: consultar, recomendar o ejecutar.
- Dar a cada especialista solo las fuentes y las herramientas que su tarea necesita.

ARTEFACTOS

- El grafo: quién hace qué y qué pasa entre ellos.
- El nivel de autonomía escrito por actividad.

QUIÉN DECIDE

El responsable del proceso; lo irreversible queda en una persona.

CONTROL QUE CIERRA LA FASE

Ninguna acción irreversible sin un punto de aprobación humana en el grafo.

> • LO QUE HICE
>
> En Vesting los tres niveles —consultar, recomendar, ejecutar— se decidían caso por caso y quedaban escritos antes de construir.

3

FASE 3

Construir con contratos

ENTRADAS

- El grafo y los niveles de autonomía.

ACTIVIDADES

- Escribir la constitución de cada agente: sus leyes, sus comandos y sus fuentes.
- Escribir criterios de aceptación que pasan o no pasan, sin juicio intermedio.
- Preparar, para cada control, un caso roto a propósito que demuestre que dispara.
- Resolver con código lo que exige exactitud: cálculos, formatos, consultas.

ARTEFACTOS

- El harness de cada especialista.
- Los criterios binarios y su carnada, el caso roto de cada control.

QUIÉN DECIDE

Quien construye, dentro del contrato.

CONTROL QUE CIERRA LA FASE

Cada control se ve fallar con su caso roto antes de darse por bueno.

> • LO QUE HICE
>
> Así nacen los trece agentes de mi vitrina que corren sobre Claude Code: una constitución en un CLAUDE.md, el harness con sus criterios binarios —20 en el Constructor de Tableros Power BI, 44 en el Harness Paper Computacional— y una carnada por control.

4

FASE 4

Evaluar antes de ponerlo en producción

ENTRADAS

- El agente construido y sus criterios.
- Escenarios habituales, excepciones conocidas y condiciones límite.

ACTIVIDADES

- Correr la batería de escenarios contra los criterios.
- Comparar contra una línea base declarada, con la misma muestra y la misma métrica.
- Comprobar que el agente se abstiene o pasa el caso cuando la evidencia no alcanza.
- Probar las transiciones entre especialistas, no solo a cada uno por separado.

ARTEFACTOS

- El informe de criterios: cuáles pasan y cuáles no.
- La comparación con la línea base: muestra, métrica y resultado.

QUIÉN DECIDE

Quien responde por el resultado, con la definición de terminado acordada al inicio.

CONTROL QUE CIERRA LA FASE

Se juzga lo que se escribió, no si el agente «se ve bien».

> • LO QUE HICE
>
> Con ARKHÉ comparé 120 escenarios contra un agente generalista que recibía todo en cada ejecución: en mis mediciones, 52 % menos tokens y el cumplimiento de instrucciones del 71 % al 93 %, sin un artefacto publicado.

5

FASE 5

Operar y aprender

ENTRADAS

- El agente publicado.
- Los eventos de cada sesión.

ACTIVIDADES

- Reconstruir la sesión completa: solicitud, contexto, respuesta, tiempo, costo y estado final.
- Mirar cuatro dimensiones a la vez: operativa, de uso, económica y funcional.
- Convertir cada falla en un caso nuevo de la batería.

ARTEFACTOS

- Tableros de operación por agente.
- Casos nuevos para la siguiente evaluación.

QUIÉN DECIDE

Producto y operación, con el tablero delante.

CONTROL QUE CIERRA LA FASE

Una señal avisa antes de que el usuario lo note.

> ● LO QUE HICE
>
> En Vesting la plataforma llegó a vigilar hasta 23 agentes a la vez, en tiempo real, con la sesión como unidad de análisis y esas cuatro dimensiones.

BLOQUE 4

Las decisiones de diseño

Un agente o un grafo de especialistas

CUÁNDO SÍ

Cuando las tareas necesitan contextos, fuentes o herramientas distintas, o se pueden hacer en paralelo.

CUÁNDO NO

Cuando todos los pasos comparten el mismo contexto o dependen mucho unos de otros: ahí repartir solo suma costo.

POR QUÉ

> ◆ LO QUE EL CAMPO HACE
>
> Según Anthropic, un sistema multiagente gasta unas quince veces los tokens de una conversación, y los dominios donde todos los agentes comparten el mismo contexto o dependen mucho entre sí hoy no le sientan bien.[1]

> ● LO QUE HICE
>
> ARKHÉ evita al agente generalista que recibe todo el conocimiento, todas las herramientas y todas las responsabilidades: reparte el trabajo entre capacidades especializadas con reglas explícitas de coordinación.

Qué es agente y qué es código

CUÁNDO SÍ

Agente, cuando la tarea necesita interpretar, sintetizar o coordinar de forma flexible.

CUÁNDO NO

Cuando la tarea exige exactitud, repetibilidad o una validación inequívoca: un cálculo, un formato, una consulta.

POR QUÉ

◆ LO QUE EL CAMPO HACE

Microsoft lo deja escrito en la documentación de su Agent Framework: si puedes escribir una función que resuelva la tarea, hazlo en vez de usar un agente.[2]

● LO QUE HICE

En el Experto Fiscal la aritmética corre en Python con Decimal y el modelo no toca una cifra; en el Constructor de Tableros Power BI cada medida DAX se prueba con una consulta antes de pasar de fase.

Dónde vive el estado

CUÁNDO SÍ

Estado guardado, cuando el trabajo dura más que una conversación, puede fallar a mitad de camino o debe esperar a una persona.

CUÁNDO NO

Cuando todo termina en una sola pasada y repetirla cuesta menos que guardarla.

POR QUÉ

◆ LO QUE EL CAMPO HACE

LangGraph 1.0 guarda el estado de la ejecución automáticamente y permite pausar para que una persona revise, modifique o apruebe, y retomar desde ahí.[3]

● LO QUE HICE

En ARKHÉ la memoria se diseña por función: decisiones que persisten, resultados reutilizables, contexto temporal y detalle que se descarta; en los agentes publicados toma forma de bitácora, lecciones de cada corrida y decisiones de arquitectura registradas.

Cuánta autonomía, y dónde se detiene ante una persona

CUÁNDO SÍ

Que ejecute solo, cuando la acción es reversible, verificable y de bajo impacto.

CUÁNDO NO

Cuando la consecuencia es significativa, difícil de revertir o poco observable: ahí el agente recomienda y una persona decide.

POR QUÉ

◆ LO QUE EL CAMPO HACE

OWASP llama agencia excesiva a darle a un agente más funciones, permisos o autonomía de los necesarios, y recomienda la aprobación humana de las acciones de alto impacto y la autorización en el sistema de destino, no en el criterio del modelo.[4]

● LO QUE HICE

En mis agentes el punto de aprobación es mecánica, no un recordatorio: el agente se detiene hasta que la persona escribe una palabra literal. El Constructor de Tableros Power BI tiene 5 de esos gates.

Del BPMN a la orquestación

CUÁNDO SÍ

Cuando el proceso ya existe y está aprobado: el diagrama es la mejor especificación inicial del grafo.

CUÁNDO NO

Cuando la tarea es abierta y no tiene pasos definidos: un diagrama inventado solo da falsa precisión.

POR QUÉ

▲ MI POSICIÓN

Un diagrama muestra la secuencia, pero no lo que cada actividad recibe, transforma y entrega. Si el proceso está aprobado, lo que corre debe trazar a él caja por caja; si no, lo aprobado y lo que corre se separan sin que nadie lo vea.

● LO QUE HICE

FORJA, una de mis investigaciones, abre cada actividad del BPMN con entradas, función y salidas y compila el diagrama a una orquestación de agentes que conserva la estructura del BPMN: en el paper sellado conservó el 99,5 %, y el único elemento de 204 que quedó sin trazar estaba fuera del subconjunto especificado.

Cómo se juzga un resultado

CUÁNDO SÍ

Criterio binario, cuando el resultado se puede comprobar: un formato, una fuente citada, una herramienta bien elegida, una abstención correcta.

CUÁNDO NO

Cuando el criterio es difuso: ahí ayuda un modelo como juez, calibrado contra el juicio de una persona y nunca como único control.

POR QUÉ

◆ LO QUE EL CAMPO HACE

Anthropic describe tres formas de que un agente verifique su trabajo —reglas, retroalimentación visual y un modelo como juez— y advierte que el juez, en general, no es un método muy robusto. [5]

▲ MI POSICIÓN

Prefiero un criterio que pasa o no pasa a una nota de 1 a 5: la nota se discute, el criterio se cumple o no.

Qué modelo para cada paso

CUÁNDO SÍ

Un modelo grande, para el paso que coordina, planea o razona sobre mucha información.

CUÁNDO NO

Para los pasos acotados y repetitivos: ahí un modelo pequeño basta y cuesta menos.

POR QUÉ

◆ LO QUE EL CAMPO HACE

El enrutador de modelos de Microsoft Foundry (antes Azure AI Foundry) usa modelos más pequeños y baratos cuando bastan y reserva los grandes para lo complejo; el sistema de investigación de Anthropic dirige con un modelo mayor y delega en subagentes de un modelo menor. [6] [1]

● LO QUE HICE

En ARKHÉ el ahorro no vino de un modelo más barato: en mis mediciones, el 52 % menos de tokens salió de mandar menos contexto inútil en cada llamada.

BLOQUE 5

Lo innovador en 2026

◆ LO QUE EL CAMPO HACE

Ejecución durable: el estado se guarda en cada paso

QUÉ CAMBIA FRENTE A LA PRÁCTICA COMÚN

Un trabajo largo sobrevive a una falla o a la espera de una persona y se retoma desde el último paso guardado, en vez de empezar de cero. LangGraph 1.0 guarda el estado automáticamente y Agent Framework 1.0 lo ofrece en sus flujos con puntos de guardado.[3][7][1]

▲ MI POSICIÓN

Mi regla: si un paso va a esperar a una persona, el estado se guarda antes de preguntarle.

◆ LO QUE EL CAMPO HACE

Ingeniería de contexto, no solo de prompts

QUÉ CAMBIA FRENTE A LA PRÁCTICA COMÚN

La pregunta deja de ser cómo redactar la instrucción y pasa a ser qué entra al contexto de cada llamada. El contexto se trata como un recurso finito: se compacta cuando se llena, se anota fuera de la ventana y se reparte entre subagentes con contexto limpio que devuelven solo un resumen.[8][5]

● LO QUE HICE

Es la idea con la que trabaja ARKHÉ: ningún componente recibe información solo porque está disponible; recibe lo que su tarea necesita, recuperado de fuentes autorizadas.

◆ LO QUE EL CAMPO HACE

Protocolos comunes: MCP para herramientas, A2A entre agentes

QUÉ CAMBIA FRENTE A LA PRÁCTICA COMÚN

Una herramienta se conecta una vez por un estándar y la usan agentes de cualquier framework; los servidores se descubren en un registro abierto, y agentes de plataformas distintas pueden hablarse. En su revisión de julio de 2026, MCP se volvió sin estado y llevó las tareas largas a una extensión oficial.[9][10][11]

● LO QUE HICE

El Constructor de Tableros Power BI crea las medidas, columnas y relaciones por el servidor MCP de modelado, probando cada medida en DAX antes de seguir.

◆ LO QUE EL CAMPO HACE

Evaluar la conversación entera y el camino, no solo la respuesta

QUÉ CAMBIA FRENTE A LA PRÁCTICA COMÚN

Lo que se prueba antes de publicar solo cubre lo que uno anticipó; por eso la evaluación sigue en producción sobre la conversación completa —si el agente logró el objetivo— y sobre el proceso paso a paso: qué herramienta eligió, con qué parámetros y qué hizo con el resultado.[12][13]

> • LO QUE HICE
>
> En Vesting la sesión ya era la unidad de análisis: se reconstruía evento por evento para ver en qué momento una interacción se había desviado.

◆ LO QUE EL CAMPO HACE

Trazas con OpenTelemetry de punta a punta

QUÉ CAMBIA FRENTE A LA PRÁCTICA COMÚN

La traza del agente viaja por el mismo estándar abierto que el resto del sistema, así que una falla se sigue desde la solicitud hasta la herramienta aunque cruce frameworks y servidores.[14][9][7]

◆ LO QUE EL CAMPO HACE

Frameworks que convergen y una plataforma que los aloja

QUÉ CAMBIA FRENTE A LA PRÁCTICA COMÚN

Microsoft unió Semantic Kernel y AutoGen en Agent Framework 1.0, con flujos en grafo, aprobaciones humanas, MCP y A2A. Microsoft Foundry aloja agentes escritos con él o con LangGraph y le da a cada uno su propia identidad, con el estado de sesión y la observabilidad gestionados.[7][15]

> • LO QUE HICE
>
> Con Microsoft Agent Framework y Claude Agent SDK experimento y construyo prototipos en la Fundación CTIC; con el ecosistema de LangChain trabajo allí junto con n8n.

◆ LO QUE EL CAMPO HACE

La agencia excesiva, riesgo de primera línea

QUÉ CAMBIA FRENTE A LA PRÁCTICA COMÚN

En la edición 2026 de su Top 10 para aplicaciones con LLM, OWASP subió la agencia excesiva del sexto al tercer lugar: darle a un agente más funciones, permisos o autonomía de los que necesita queda entre los tres riesgos primeros de la lista.[16][4]

BLOQUE 6

Modos de falla y cómo se ven venir

QUÉ SE ROMPE	CÓMO SE VE VENIR	CONTROL QUE LA ATRAPA
El agente responde, pero no resuelve	Sesiones que terminan sin el estado esperado o con una persona interviniendo.	La dimensión funcional del monitoreo: estados finales y sesiones que necesitaron intervención, por agente.
El contexto se llena de ruido	Suben los tokens por tarea completada sin que suba el cumplimiento.	Tokens por tarea completada medidos junto con el cumplimiento de instrucciones, contra la línea base.
Un especialista le entrega al siguiente algo incompleto	Cada pieza pasa sus pruebas y el conjunto falla.	Contratos de entrada y salida entre especialistas, con pruebas de la transición y no solo de cada uno.
El agente hace más de lo que le tocaba	Una acción con consecuencias aparece en las trazas sin una aprobación al lado.	Permisos mínimos por especialista y un punto de aprobación con palabra literal antes de lo irreversible.
El modelo inventa una cifra o una fuente	Una afirmación verificable sin el fragmento que la sostiene.	Fuente autorizada o vacío declarado: un validador rechaza la salida que cita lo que no existe.

QUÉ SE ROMPE	CÓMO SE VE VENIR	CONTROL QUE LA ATRAPA
Un control que nunca falla	Una prueba siempre en verde que nadie ha visto ponerse en rojo.	Una carnada por control: el caso roto a propósito que demuestra que dispara.

QUÉ SE ROMPE	CÓMO SE VE VENIR	CONTROL QUE LA ATRAPA
El costo crece en silencio	Reintentos, sesiones abandonadas o relecturas masivas que no producen nada.	Costo en tokens por sesión y por agente, visto junto a lo que resuelve, con un umbral escrito para cada patrón.

QUÉ SE ROMPE	CÓMO SE VE VENIR	CONTROL QUE LA ATRAPA
Una instrucción entra por una herramienta	Un texto leído de una página o de un documento le pide algo al agente.	Lo que dice una fuente es dato, no instrucción: se ignora, se registra, y la autorización vive en el sistema de destino.

BLOQUE 7

Métricas y controles

QUÉ SE MIDE	UMBRAL	QUIÉN APRUEBA EL PASO A PRODUCCIÓN
Criterios de aceptación cumplidos	`Todos en verde: un criterio binario no se negocia.`	Quien responde por el resultado, contra la definición de terminado acordada al inicio.

QUÉ SE MIDE	UMBRAL	QUIÉN APRUEBA EL PASO A PRODUCCIÓN
Controles con una carnada que dispara	`Todos los controles críticos; el que no la tenga se declara como tal.`	Quien construye, revisado por alguien que no construyó.

QUÉ SE MIDE	UMBRAL	QUIÉN APRUEBA EL PASO A PRODUCCIÓN
Cumplimiento de instrucciones en la batería de escenarios	`Igual o mejor que la línea base declarada, sobre la misma muestra.`	Quien responde por el resultado.

QUÉ SE MIDE	UMBRAL	QUIÉN APRUEBA EL PASO A PRODUCCIÓN
Tokens por tarea completada	No peor que la línea base, y nunca a costa del cumplimiento.	Quien paga la operación.
Abstenciones correctas	En cada caso sin evidencia, el agente declara el vacío o pasa el caso; un solo dato inventado frena el paso.	El responsable del proceso.
Sesiones que necesitaron intervención humana	Por debajo del tope acordado para la versión; si lo pasa, la versión vuelve a evaluación.	Producto y operación, con el tablero delante.
Acciones irreversibles sin aprobación	Cero.	El responsable del proceso.

Los umbrales son ejemplos del método, no cifras de un proyecto mío.

BLOQUE 8

Mi evidencia

• LO QUE HICE USO REAL

Vesting: 27 agentes en producción

Definí, documenté y validé el proceso core de once etapas con el que una startup construyó 27 agentes para 12 clientes sobre n8n, y diseñé en Microsoft Fabric la plataforma que llegó a vigilar 23 a la vez, en tiempo real.

Ver dónde →

• LO QUE HICE USO REAL

ARKHÉ: mi ecosistema agéntico

Harnesses especializados que fijan a cada agente función, fuentes, herramientas y contrato. En mis mediciones sobre 120 escenarios, frente a un generalista: 52 % menos tokens y cumplimiento de instrucciones del 71 % al 93 %. Es de uso personal; no lo llamo producción.

Ver dónde →

• LO QUE HICE PROTOTIPO

FORJA: del BPMN a una orquestación de agentes

FORJA es una de mis investigaciones: un compilador que convierte un proceso dibujado en BPMN en una orquestación de agentes. Del BPMN conserva la estructura —cada actividad se abre con entradas, función y salidas, y cada elemento traza a su realización—: en el paper sellado conservó el 99,5 %. Verificadores de distintas familias vigilan la conformidad. El demostrador existe.

Ver dónde →

• LO QUE HICE USO REAL

Los agentes de la vitrina, sobre Claude Code

Trece sistemas de trabajo con constitución, harness, criterios binarios, carnadas y gates humanos con palabra literal; cinco están sellados. Son de uso personal, con entregables y evidencia.

Ver dónde →

• LO QUE HICE USO REAL

Constructor de Tableros Power BI

Crea medidas, columnas y relaciones por el servidor MCP de modelado, probando cada medida en DAX, y escribe el reporte en el JSON nativo de Power BI; 20 criterios y 5 gates humanos, y ninguna corrida cierra sin el render aprobado por una persona.

Ver dónde →

• LO QUE HICE USO REAL

Experto ISO 42001

Cada afirmación normativa cita apartado y página o declara el vacío, y ningún dictamen con consecuencias avanza sin la aprobación de una persona. Es el que uso en la estrategia de IA de la Fundación CTIC.

Ver dónde →

• LO QUE HICE USO REAL

LangChain, LangGraph, LangSmith y Langflow

Son, con n8n, los frameworks de agentes con los que trabajo en la Fundación CTIC. Ese trabajo no tiene un entregable publicado en este sitio.

Ver dónde →

• LO QUE HICE PROTOTIPO

Microsoft Agent Framework y Claude Agent SDK

Con ellos experimento y construyo prototipos en la Fundación CTIC.

• LO QUE HICE USO REAL

El chat de este sitio

Recupera del corpus, cita lo que recuperó y, si no encuentra, lo dice. Desde septiembre de 2026 guarda una fila por respuesta: quién preguntó, qué se contestó, con qué fuentes, con qué proveedor, cuántos tokens y cuántos milisegundos.

BLOQUE 9

Lo que haría distinto y lo que no he hecho

> • LO QUE HICE
>
> La palabra «producción» la reservo para los 27 agentes de Vesting. Los de ARKHÉ y los de la vitrina son sistemas de trabajo de uso personal: no han tenido usuarios distintos de mí.

• LO QUE HICE

Las cifras de ARKHÉ son mis propios cálculos, sin un artefacto publicado: miden menos contexto inútil y más obediencia a la especificación, no la calidad en abstracto, y no sirven para comparar con otro ecosistema sin repetir los mismos escenarios.

• LO QUE HICE

Con Microsoft Agent Framework y Claude Agent SDK he hecho experimentación y prototipos, no producción. Con el ecosistema de LangChain trabajo en la Fundación CTIC, sin un entregable publicado aquí.

• LO QUE HICE

No he operado agentes en contenedores: Docker y Kubernetes en producción no están en mi experiencia profesional.

• LO QUE HICE

Con Microsoft Foundry, mi experiencia es el adaptador de proveedor del chat de este sitio y el estudio del AI-103, que está en curso; un sistema de agentes alojado allí no está en mi experiencia.

• LO QUE HICE

FORJA tiene su demostrador —compilador, cinco procesos, línea base y compuertas de aprobación—, pero que los profesionales lo usen se demuestra con profesionales, y esa prueba falta.

▲ MI POSICIÓN

Lo que haría distinto hoy: trazas con un estándar abierto desde el primer agente, para seguir una sesión aunque cruce frameworks, y la conversación entera evaluada en línea, no solo antes de publicar.

▲ MI POSICIÓN

Y no empezaría por un grafo de muchos agentes: empezaría con uno, lo mediría contra una línea base y solo repartiría el trabajo donde el contexto o el paralelismo lo justifiquen.

BLOQUE 10

Fuentes y fecha de revisión

Revisado el 17 de octubre de 2026

[1] How we built our multi-agent research system

Anthropic · 13 de junio de 2025

https://www.anthropic.com/engineering/multi-agent-research-system

[2] Microsoft Agent Framework Overview

Microsoft Learn · 29 de julio de 2026

https://learn.microsoft.com/en-us/agent-framework/overview/

[3] LangChain and LangGraph Agent Frameworks Reach v1.0 Milestones
LangChain · 22 de octubre de 2025
https://www.langchain.com/blog/langchain-langgraph-1dot0

[4] LLM06:2025 Excessive Agency (OWASP Top 10 for LLM Applications 2025)
OWASP GenAI Security Project · 17 de noviembre de 2024
https://genai.owasp.org/llmrisk/llm062025-excessive-agency/

[5] Building agents with the Claude Agent SDK
Anthropic · 29 de septiembre de 2025
https://claude.com/blog/building-agents-with-the-claude-agent-sdk

[6] Model router for Microsoft Foundry concepts
Microsoft Learn · 1 de septiembre de 2026
https://learn.microsoft.com/en-us/azure/foundry/openai/concepts/model-router

[7] Microsoft Agent Framework Version 1.0
Microsoft · 3 de abril de 2026
https://devblogs.microsoft.com/agent-framework/microsoft-agent-framework-version-1-0/

[8] Effective context engineering for AI agents
Anthropic · 29 de septiembre de 2025
https://www.anthropic.com/engineering/effective-context-engineering-for-ai-agents

[9] Model Context Protocol specification 2026-07-28: Key Changes
Model Context Protocol · 28 de julio de 2026
https://modelcontextprotocol.io/specification/2026-07-28/changelog

[10] Introducing the MCP Registry
Model Context Protocol · 8 de septiembre de 2025
https://blog.modelcontextprotocol.io/posts/2025-09-08-mcp-registry-preview/

[11] Linux Foundation Launches the Agent2Agent Protocol Project to Enable Secure, Intelligent Communication Between AI Agents
Linux Foundation · 23 de junio de 2025
https://www.linuxfoundation.org/press/linux-foundation-launches-the-agent2agent-protocol-project-to-enable-secure-intelligent-communication-between-ai-agents

[12] Improve agent quality with Insights Agent and Multi-turn Evals, now in LangSmith
LangChain · 23 de octubre de 2025
https://www.langchain.com/blog/insights-agent-multiturn-evals-langsmith

[13] Agent Evaluators for Generative AI
Microsoft Learn · 30 de septiembre de 2026
https://learn.microsoft.com/en-us/azure/foundry/concepts/evaluation-evaluators/agent-evaluators

[14] Introducing End-to-End OpenTelemetry Support in LangSmith
LangChain · 26 de marzo de 2025
https://www.langchain.com/blog/end-to-end-opentelemetry-langsmith

[15] Hosted agents in Foundry Agent Service
Microsoft Learn · 11 de septiembre de 2026
https://learn.microsoft.com/en-us/azure/foundry/agents/concepts/hosted-agents

[16] OWASP GenAI Security Project Unveils 2026 Top 10 for LLM Applications, New Agent Control Standard and Sponsors as Community Tops 30,000 Members

OWASP GenAI Security Project · 1 de septiembre de 2026

https://genai.owasp.org/announcement/owasp-genai-security-project-unveils-2026-top-10-for-llm-applications-new-agent-control-standard-and-sponsors-as-community-tops-30000-members/